

Teaching AI to Speak Design System

How I built a context pipeline that makes AI generate DS-compliant UI — and what it means for how product teams work

Role: Interaction Designer

Company: Western Union (simulated on Material Design 3)

Tools: Figma, Figma MCP, Claude, VS Code, Storybook, React, GitHub

Type: Internal tooling / Design system infrastructure

Timeline: Self-initiated research project

The Problem Nobody Was Saying Out Loud

Every designer on a product team has felt this. You spend weeks refining the design system — the right tokens, the right component variants, the right usage rules. You document it carefully. You present it to the team. Everyone nods.

Then a developer uses AI to scaffold a screen and it comes out looking like a Bootstrap template from 2016.

At a company like Western Union — where global scale means dozens of product teams, thousands of screens, and a design system that needs to stay consistent across all of them — this isn't just an aesthetic problem. It's a trust problem. Engineers stop trusting the DS because the tools they use ignore it. Designers stop trusting the handoff because they can't predict what will come out. Product managers stop trusting timelines because design reviews keep catching regressions.

I'd watched this pattern play out for years. AI code generation was getting better, faster, and more integrated into engineering workflows. But the outputs kept missing the point. Not because the AI was bad at writing code. Because nobody had told it what *our* code was supposed to look like.

The problem wasn't AI capability. The problem was context.

The Insight That Reframed Everything

Most conversations about AI in design teams focus on what AI can do: generate wireframes, suggest copy, produce code. That's the wrong frame. It puts AI in the role of the creator and the designer in the role of the reviewer.

I started asking a different question: **what does AI need from design in order to be useful?**

The answer was obvious once I said it out loud. AI generates generic output because it has no access to the decisions that make our product ours. It doesn't know that we use a 4pt spacing scale, or that filled buttons are reserved for primary CTAs, or that navigation on mobile means a bottom nav bar with 3-5 items and never a hamburger menu. It doesn't know any of this because nobody ever told it. No one had built the bridge.

What if the design system wasn't just documentation for humans? What if it was structured context that AI could actually read and act on?

That became the project.

What I Set Out to Build

Not a tool. Not a plugin. A pipeline.

The distinction matters. A tool is a single step — you do something, it produces an output. A pipeline is a system — every part informs every other part, and the output at the end is only as good as the context flowing through the whole thing.

I wanted a pipeline where:

- Figma tokens and components were readable by AI tools via MCP
- Design system rules were structured as machine-readable context
- Code components directly mirrored Figma components in naming and variants
- Every piece of generated code could be traced back to a DS decision
- The system maintained itself — new decisions propagated everywhere

And critically: I wanted to build it without touching the real production codebase. So I ran the entire experiment on Material Design 3, Google's open design system, as a proxy. Same structural challenge, same token complexity, same component vocabulary — but a safe sandbox where I could move fast and test everything.

The Design Process

Phase 1 — Understanding the failure modes

Before building anything, I spent time cataloguing exactly how and where AI generation went wrong. I ran identical prompts across different conditions and documented the outputs systematically.

Without context:

- Hardcoded hex values (`color: #6750A4`) instead of token references
- Raw HTML div layouts instead of design system components
- Arbitrary spacing (10px, 15px, 22px) instead of the 4pt scale
- Wrong component choices for context (e.g. hamburger nav on mobile)
- No semantic role applied to color — primary used for decoration

With partial context (just a Figma link):

- Better visual resemblance but still no token usage
- Component names guessed rather than matched to the DS
- States and variants chosen arbitrarily

The gap was consistent: AI had no access to the *rules* behind the design, only the *appearance* of the output.

This told me something important: the problem wasn't primarily in the AI. It was in the absence of structured, machine-readable design intent. Fixing this was a design problem before it was a technical one.

Phase 2 — Designing the context layer

I defined four layers that would need to exist for the pipeline to work:

Layer 1: Token documentation Every design decision at the token level had to be explicit, named, and connected to a semantic role. Not just "the primary color is #6750A4" but "primary is used for key interactive actions, paired always with on-primary for text — never used decoratively."

Layer 2: Component rules Each component needed a clear decision tree. Not just what variants exist, but when to use each one. Which variant is the default? Which is reserved for destructive actions? What's the maximum number allowed on a single screen?

Layer 3: Layout patterns Screen-level decisions needed to be codified as repeatable patterns. A dashboard screen looks like X. A form screen looks like Y. A detail screen looks like Z. These aren't constraints — they're the accumulated design thinking of the team, made legible to a machine.

Layer 4: Code-side enforcement Context files alone aren't enough. The code layer needed to make wrong usage structurally impossible. TypeScript wrapper components that only accept valid variants. Import paths that force usage through the DS layer. A theme file where every spacing value is a multiple of 4 because that's the only option exposed.

Phase 3 — Building the inventory

To document what existed in the design system, I used Figma MCP to systematically read every page of the M3 Design Kit. Not manually — I used AI to traverse the file and produce two structured outputs:

- A markdown inventory (`material-3-design-kit-inventory.md`) — human-readable, 23+ pages, every component group and variant axis
- A JSON variant map (`material-3-design-kit-variants.json`) — machine-readable, 2,129 lines, validated at 29/29 pages with zero missing groups

This inventory became the single source of truth for everything built after it. Not a spec written from memory. A direct, verified extraction from the design system itself.

Phase 4 — Building the context files

I wrote `design-system.md` — the core context document that AI tools read before generating anything. It covers:

- Token conventions (semantic roles, spacing scale, shape values)
- Component selection rules with explicit decision criteria
- Screen layout patterns with component-level descriptions
- Hard rules (no hardcoded values, max one filled button per screen, etc.)

This is the file that changed output quality more than anything else. Before it existed, AI was guessing. After it existed, AI was following documented design decisions.

I also wrote `.github/copilot-instructions.md` — a shorter, codebase-specific version that the AI reads automatically every time a file in the repo is open. It reinforces the rules at the moment of generation.

Phase 5 — The component library

Working from the inventory, I built 25 TypeScript wrapper components — one for every component family in the design system. Each wrapper:

- Accepts only the variants that exist in Figma (enforced by TypeScript union types)
- Uses only theme tokens for all visual values
- Is named identically to its Figma counterpart
- Includes a comment stating the M3 rule it enforces

This is where the design and code truly meet. When a developer (or an AI) uses `DSButton variant="filled"`, they can't accidentally pass `variant="primary"` — TypeScript won't compile. The design rules are enforced at the type level, not the review level.

Phase 6 — Storybook as the living DS documentation

I set up Storybook not as a component playground but as a full documentation site. Each of the 25 components has:

- An MDX documentation page with overview, variants, states, Do's/Don'ts, token reference table, accessibility notes, and related component links
- An args-based story set covering every variant and state from the Figma inventory
- A props table that reflects the TypeScript interface directly

The token documentation pages — Color, Typography, Shape — show the full M3 palette with live swatches, typescale with live previews, and shape scale with component mappings.

The intentional gap: I deliberately left 90 accessibility placeholder lines across all 25 components. Not an oversight — a demonstration. This is Part 3 of the demo: showing that a single prompt can fill 90 documentation gaps across 25 components in real time, sourced from M3 spec and WCAG 2.1 AA guidelines.

What the Pipeline Produces

The test is simple. Same prompt, twice. First without the context pipeline active. Then with it.

Without:

```
<div style={{ backgroundColor: '#6750A4', padding: '10px' }}>
  <h2>Dashboard</h2>
</div>
```

```
    <div className="card">...</div>
  </div>
```

Hardcoded. Generic. No DS. Would be rejected in any design review.

With:

```
<DSAppBar size="small" title="Dashboard" rightElement={<Avatar />} />
<Box sx={{(theme) => ({ padding: theme.spacing(4) })}}>
  <Box sx={{ display: 'grid', gridTemplateColumns: 'repeat(2, 1fr)',
    gap: (theme) => theme.spacing(3) }}>
    {stats.map(stat => (
      <DSCard variant="elevated">
        <CardContent>...</CardContent>
      </DSCard>
    ))}
  </Box>
</Box>
<DSNavigationBar items={navItems} activeIndex={0} onChange={setNav} />
```

M3 purple applied through tokens. Spacing from the 4pt scale. Correct component for each context. BottomNavigation for mobile. No hardcoded values anywhere.

This isn't AI getting lucky. This is AI following the design decisions I built into the pipeline.

The Bigger Picture: What This Actually Changes

For designers

Design decisions stop dying in handoff. The reasoning behind a token, a variant rule, a layout pattern — it now persists as structured context that flows into every generation. The design system becomes genuinely authoritative, not just aspirational.

For developers

Reduced interpretation overhead. Instead of reading a spec and translating it manually, a developer pastes a Figma link and gets DS-compliant code as a starting point. The pipeline handles the translation. Human judgment handles what matters: product logic, edge cases, accessibility.

For product and business stakeholders

This was the least expected outcome, but potentially the most valuable. Non-designers can

now prompt-generate rough screens for brainstorming, user testing concepts, or stakeholder reviews — without knowing Figma. The output is DS-compliant by construction. This democratises the early stages of design without democratising design quality downward.

For the design system itself

It stops being a document that people consult and becomes an input that tools consume. The DS becomes more valuable the more AI is used — not less. This inverts the usual fear around AI displacing design work.

Reflections on the Process

What I got wrong first: I initially thought better prompts would fix the problem. They don't. Prompts are ephemeral — they don't persist, they don't scale, they don't enforce anything. Structured context that lives in the codebase is fundamentally different from a well-written prompt.

What surprised me: How much the TypeScript enforcement matters. It's easy to underestimate this as a "developer concern." It isn't. When the type system only accepts valid DS variants, it means the design decisions are enforced at compile time, not code review time. That's a different category of reliability.

What remains hard: Code Connect — the piece that would make Figma Dev Mode show real codebase components when designers inspect — is blocked by a Figma API limitation on community file duplicates. This is the missing link that would close the loop from design to dev view. It requires an owned, non-community Figma file to publish properly.

What I'd do differently at scale: Run this with our real design system from the start, not a proxy. The proxy was the right move for a research project, but the migration path is clear and documented. The structural decisions are identical.

What's Next

The simulation is complete. The migration guide exists. The pending question is adoption.

The next version of this pipeline applied to our real DS would include:

- Code Connect publishing from our own Figma file, so every component in Dev Mode shows live code

- Exhaustive variant story generation — 140+ stories per component for visual regression testing
 - AI-assisted accessibility auditing using the same prompt infrastructure
 - Extension to mobile and web variants of the DS, not just the core component set
-

What I Learned About Designing for AI

AI is not a replacement for design thinking. It is an amplifier of design thinking — and the quality of what it amplifies depends entirely on the quality of the context you give it.

Building this pipeline taught me that the most important design work of the next decade won't be designing for users. It will be designing the systems that make AI useful to teams. That means structured documentation, machine-readable rules, token-level precision, and an understanding of how information flows between tools.

That's interaction design. Applied to a new kind of interface.

Appendix: What Was Built

Deliverable	Detail
DS context file	<code>design-system.md</code> — token rules, component rules, layout patterns
AI instructions	<code>.github/copilot-instructions.md</code> — codebase-specific AI rules
Figma inventory	<code>material-3-design-kit-inventory.md</code> — all 23 component pages
Variant map	<code>material-3-design-kit-variants.json</code> — 2,129 lines, machine-readable
DS wrappers	25 TypeScript components, all variants enforced
Storybook stories	25 story files, args-based, autodocs enabled
MDX documentation	25 component doc pages + 3 token pages
Token documentation	<code>color.md</code> , <code>typography.md</code> , <code>shape.md</code> with usage rules

Demo screens	Dashboard.tsx, Settings.tsx — generated, DS-compliant
GitHub repo	github.com/kushagra-wu/material-ds-simulation

This case study documents a self-initiated research project. The pipeline was built and validated on Material Design 3 as a proxy system. All technical implementation was executed using AI tools (Claude, Codex) orchestrated by the designer — no engineering support was required. The design thinking, system architecture, and strategic framing are the designer's own work.